Introduction to Python for Finance

Comprehensive Review — All Topics

print, input, if, loops, functions, lists, DataFrames

Instructions: this sheet reviews everything seen in class, from basic print statements to pandas DataFrames. Work through the sections in order — later exercises reuse skills from earlier ones. Functions must return their result unless an exercise explicitly says to print.

Section 1 — print, input, if

Goal: warm up with the basics before combining them with loops and functions.

Exercise 1 — Account Login Check

Ask the user to enter a password with input().
If it equals "finance2026", print "Access granted." Otherwise print "Access denied."

> **Tip:** String comparison with == is case-sensitive: "Finance2026" and "finance2026" are different values.

Exercise 2 — Classify a Transaction

Create a variable amount = -250.
Print "Deposit" if amount > 0, "Withdrawal" if amount < 0, or "No transaction" if it equals 0.

Exercise 3 — Loan Eligibility

Ask the user for their monthly income and their monthly debt payments (two input() calls, converted to float).
Compute the debt ratio: debt / income.
If the ratio is below 0.35, print "Eligible". Otherwise print "Not eligible".

> **Tip:** Convert both inputs to float before dividing, or Python will try to divide two strings and raise an error.

Exercise 4 — Portfolio Health Check

Create variables cash = 2000 and invested = 8000.
Compute total = cash + invested, and cash_ratio = cash / total * 100.
Print "Low cash buffer" if cash_ratio < 10, "Balanced" if it is between 10 and 30, or "Cash heavy" otherwise.

Section 2 — Loops

Goal: repeat actions with for and while, on data you'll reuse in the functions and lists sections.

Exercise 6 — Print Each Ticker

tickers = ["AAPL", "MSFT", "GOOG", "AMZN"]
Use a for loop to print each ticker on its own line.

Exercise 7 — Sum of Expenses

expenses = [120, 340, 75, 560]

Use a for loop and an accumulator variable to compute and print the total.

> **Tip:** Start with total = 0 before the loop, then update it with total += e inside the loop.

Exercise 8 — Validate a Positive Input

Use a while loop to repeatedly ask the user "Enter an investment amount: " until they enter a number strictly greater than 0.

Print "Accepted:" and the amount once it is valid.

> **Tip:** Convert with float(input(...)) inside the loop, and re-check the condition on every pass.

Exercise 9 — Monthly Growth Simulation

Start with balance = 500 and rate = 0.02.

Use a while loop that multiplies balance by (1 + rate) once per month until balance reaches or exceeds 1000.

Print the number of months needed.

> **Tip:** Make sure balance actually changes inside the loop, otherwise the condition never becomes False.

Section 3 — Functions

Goal: wrap the logic from Sections 1 and 2 into reusable functions.

Exercise 10 — Interest Calculator Function

Define a function interest(principal, rate, years) that returns principal * rate * years.

Call it with 2000, 0.04 and 5, and print the result.

Exercise 11 — Function with a Default Parameter

Define a function apply_fee(amount, fee_rate=0.02) that returns amount after subtracting the fee: amount * (1 - fee_rate).

Call it once without fee_rate, and once with fee_rate=0.05.

Exercise 12 — Function Returning Multiple Values

Define a function stats(values) that returns two values: the average of the list, and its range (max - min).

You may use the built-in sum(), len(), max() and min() functions here.

Test it with returns = [0.01, -0.02, 0.03, 0.015], and unpack the two results into separate variables.

> **Tip:** A function can return several values separated by a comma: return avg, spread. Unpack with avg, spread = stats(returns).

Exercise 13 — Function Combining a Loop and a Condition

Define a function count_above(values, threshold) that returns how many elements of the list are strictly greater than threshold.

Test it with daily_pnl = [50, -20, 30, -10, 0, 75] and threshold = 0.

Section 4 — Lists

Goal: build and transform lists, the structure that feeds directly into DataFrames in Section 5.

Exercise 14 — Build a List of Squared Returns

returns = [0.01, -0.02, 0.03, 0.015]

Using a for loop, build a new list called squared where each element is the square of the corresponding return. Print it.

> **Tip:** Create an empty list first (squared = []), then use squared.append(r ** 2) inside the loop.

Exercise 15 — Filter a List of Prices

prices = [98, 150, 42, 230, 15]

Using a for loop and an if, build a new list containing only the prices above 50. Print it.

Exercise 16 — Merge Two Lists into Pairs

tickers = ["AAPL", "MSFT", "GOOG"]

prices = [189.3, 320.5, 2750.0]

Using for i in range(len(tickers)), print each pair as: AAPL: 189.3

> **Tip:** Python also has a built-in way to walk two lists together: for t, p in zip(tickers, prices). Either approach is fine here.

Section 5 — DataFrames (pandas)

Goal: move from plain lists to pandas DataFrames, the structure you'll use for real datasets. These four exercises build on the same table.

Exercise 17 — Create a DataFrame from a Dictionary

Import pandas as pd.

Build a dictionary with the keys Ticker, Sector, Price and Shares, using the data below.

Convert it into a DataFrame called df with pd.DataFrame(), and print it.

```
data = {
    "Ticker": ["AAPL", "MSFT", "GOOG", "XOM", "CVX"],
    "Sector": ["Tech", "Tech", "Tech", "Energy", "Energy"],
    "Price": [189.3, 320.5, 142.0, 110.2, 158.7],
    "Shares": [50, 20, 30, 40, 15],
}
```

Exercise 18 — Select and Filter

Using the df from Exercise 17:

Print only the Price column.

Print only the rows where Price is above 150.

> **Tip:** Select a column with df["Price"]. Filter rows with a condition inside brackets: df[df["Price"] > 150].

Exercise 19 — Add a Computed Column

Using the same df, add a new column called Value equal to Price multiplied by Shares.
Print the updated DataFrame.

> **Tip:** df["Value"] = df["Price"] * df["Shares"] works on the whole column at once — no loop needed.

Exercise 20 — Group and Aggregate

Using the df from Exercise 19 (with the Value column), compute the average Value per Sector.
Print the result.

> **Tip:** df.groupby("Sector")["Value"].mean() groups the rows by Sector first, then averages Value within each group.